

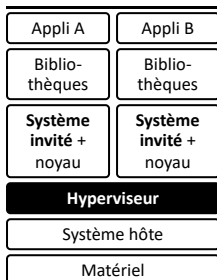
VM & Docker

Machines virtuelles · VirtualBox · KVM · WSL · conteneurs · images · réseaux · Compose · sécurité

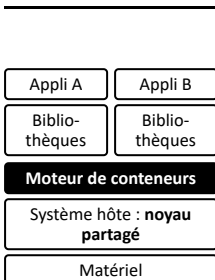
docker ps chasse fixe, gras : à taper tel quel
conteneur italique : valeur à remplacer
Ctrl+C touche ou combinaison de touches
❗ piège fréquent ou commande destructrice

DEUX FAÇONS D'ISOLER

MACHINES VIRTUELLES



CONTENEURS



La VM simule une **machine entière** ; le conteneur isole des **processus** qui partagent le noyau de l'hôte.

VM OU CONTENEUR ?

	VM	CONTENEUR
Isole	une machine complète	des processus
Noyau	le sien	celui de l'hôte
Systèmes	tout système (Windows, Linux, BSD...)	même famille : conteneur Linux ⇒ noyau Linux
Démarrage	dizaines de secondes	de l'ordre de la seconde
Taille	plusieurs Go	de quelques Mo à quelques centaines
Isolement	fort	plus faible (noyau commun)
Pour	tester un système, laboratoire, code douteux	livrer une application et ses dépendances

QUE CHOISIR ?

- Essayer un autre système, garder un poste propre → **VM**.
- Laboratoire de sécurité, programme douteux → **VM** isolée, instantané, réseau coupé.
- Faire tourner une application identique partout → **conteneur**.
- Application + base de données + cache → **Docker Compose**.
- Outils Linux sous Windows au quotidien → **WSL 2**.
- Les deux se combinent : Docker **dans** une VM Linux est courant.

SE PROCURER ISO ET IMAGES

- ISO : site **officiel** du système, empreinte vérifiée.
- Docker Hub : badges **Docker Official Image** et **Verified Publisher** ; regarder la date de mise à jour et le Dockerfile.
- ❗ VM « toutes prêtes » (OVA) et images d'inconnus : porte dérobée ou mineur de cryptomonnaie possibles.
- docker pull** n'exécute rien ; **docker run**, si.

VOCABULAIRE

Hôte	machine physique et son système, qui accueille
Invité	système qui tourne dans la VM
Hyperviseur	logiciel qui partage le matériel entre VM
Image ISO	disque d'installation d'un système
Disque virtuel	fichier vu par l'invité comme un disque
Instantané	état figé d'une VM, pour y revenir (<i>snapshot</i>)
Additions invité	pilotes d'intégration hôte ↔ invité
Image	modèle en lecture seule , en couches, d'où naissent les conteneurs
Conteneur	instance d'une image en cours d'exécution ; jetable
Couche	strate de fichiers d'une image, partagée entre images
Registre	serveur d'images (Docker Hub, GitHub...)
Volume	stockage qui survit au conteneur
Dockerfile	recette de construction d'une image
Compose	description de plusieurs conteneurs en YAML
VT-x / AMD-V	virtualisation matérielle du processeur (Intel / AMD)
Émulation	imiter un autre processeur (lent) ; la virtualisation exécute directement

LES HYPERVISEURS

Type 1	sur le matériel, sans système hôte	Hyper-V, VMware ESXi, Xen, Proxmox VE (KVM)
Type 2	application d'un système hôte	VirtualBox, VMware Workstation, Parallels
KVM	module du noyau Linux, piloté par QEMU	souvent classé type 1

Moteurs de conteneurs : **Docker**, **Podman** (sans démon), **containerd** ; orchestration à grande échelle : **Kubernetes**.

RÈGLES D'OR

- Un **instantané** avant toute expérience.
- Un instantané **n'est pas** une sauvegarde.
- Un conteneur est **jetable** : les données vont dans un **volume**.
- Fixer les versions : **postgres:17**, pas **postgres:latest**.
- Images **officielles** ou d'éditeurs vérifiés.
- Jamais de secret dans une image.

SOUS WINDOWS 11 FAMILLE

- VirtualBox** : fonctionne (hyperviseur de type 2).
- WSL 2** : disponible — un vrai noyau Linux dans une VM légère.
- Hyper-V** (gestionnaire) et **Bac à sable Windows** : réservés aux éditions Pro, Entreprise, Éducation.
- Docker Desktop** : les prérequis publiés citent Windows 11 **Pro, Entreprise ou Éducation** 23H2 ; une note y évoque les éditions Famille, limitées aux conteneurs Linux → à **vérifier** avant d'installer.
- Solution sûre : **Docker Engine dans une VM Linux**.
- ❗ Si WSL 2 est actif, VirtualBox passe par la plateforme de l'hyperviseur Windows : plus lent, parfois instable.

LA VIRTUALISATION EST-ELLE ACTIVE ?

Windows	Gestionnaire des tâches → Performances → Processeur → « Virtualisation : Activé »
Linux	<pre>grep -Ec '(vmx svm)' /proc/cpuinfo</pre> : résultat > 0 = processeur compatible <pre>lsmod grep kvm</pre> : module KVM chargé ?
Ubuntu	kvm-ok : diagnostic complet (paquet cpu-checker)

Désactivée ? Réglages UEFI (BIOS) au démarrage : **Intel VT-x** (« Virtualization Technology ») ou **AMD-V** (« SVM Mode »).

REPÈRES (SEPTEMBRE 2026)

Docker Engine	29.8.1 (15 septembre 2026) · libre (Apache 2.0)
Docker Desktop	gratuit, sauf dans une entreprise de plus de 250 salariés ou de plus de 10 M\$ de chiffre d'affaires : abonnement
VirtualBox	7.2.20 · GPL v3 ; <i>Extension Pack</i> : licence PUEL, usage personnel et éducatif
WSL	2.1.5 ou plus récent exigé par Docker Desktop

Versions vérifiées sur les sites officiels le 29 septembre 2026 : elles évoluent vite.

PROCESSEUR : PC OU ARM ?

- VM et images visent une **architecture** : **amd64** (x86-64, PC courants) ou **arm64** (Mac à puce Apple, Raspberry Pi, certains portables).
- Images **multi-architectures** : Docker choisit seul la bonne variante.
- Architecture étrangère : **émulation** possible (QEMU), mais lente.
- Connaître la sienne : **uname -m** (Linux) · **\$env:PROCESSOR_ARCHITECTURE** (PowerShell).

2 · Machines virtuelles

MODES RÉSEAU D'UNE VM (NOMS DE VIRTUALBOX)

MODE	INTERNET	HÔTE → VM	VM ↔ VM	RÉSEAU → VM	USAGE TYPE
NAT (défaut)	oui	redirection de ports	non	non	naviguer, installer ; l'invité est 10.0.2.15 , l'hôte 10.0.2.2
Réseau NAT	oui	redirection de ports	oui	non	plusieurs VM qui se voient et sortent sur Internet
Accès par pont	oui	oui	oui	oui	VM = une machine de plus sur le réseau local (adresse de la box)
Réseau privé hôte	non	oui	oui	non	serveur de test joignable depuis l'hôte seul (souvent 192.168.56.x)
Réseau interne	non	non	oui	non	laboratoire de sécurité : VM isolées entre elles
Non connecté	non	non	non	non	analyse d'un fichier suspect, hors ligne

Redirection NAT : port **hôte 2222** → port **invité 22**, puis `ssh -p 2222 utilisateur@127.0.0.1`. **!** En mode pont, la VM est exposée au réseau local comme un vrai poste : la mettre à jour.

CRÉER UNE VM : LES ÉTAPES

1. Télécharger l'**image ISO** sur le site **officiel** du système.
2. Vérifier son empreinte : `Get-FileHash f.iso` (PowerShell) ou `sha256sum f.iso`, à comparer à celle publiée.
3. Nouvelle machine : nom, type et version du système (64 bits).
4. Mémoire vive, processeurs, disque virtuel (voir ci-dessous).
5. Insérer l'ISO dans le **lecteur optique virtuel**, démarrer, installer.
6. Mettre à jour l'invité, installer les **additions invité**.
7. Éteindre proprement, prendre un instantané « **propre** ».

DIMENSIONNER

Mémoire	Linux sans bureau : 1 à 2 Go · avec bureau : 4 Go · Windows 11 : 4 Go minimum ; laisser au moins la moitié à l'hôte
Processeurs	2 suffisent souvent ; ne jamais dépasser le nombre de cœurs physiques
Disque	allocation dynamique ; 25 Go pour un Linux de bureau, 64 Go minimum pour Windows 11
Windows 11	exige aussi TPM 2.0 et démarrage sécurisé (virtuels depuis VirtualBox 7)
Affichage	mémoire vidéo au maximum si l'interface graphique rame

ÉTATS ET ARRÊT

Arrêt ACPI	demande polie : l'invité s'éteint proprement
Extinction forcée	= débrancher la prise : risque de corruption
Enregistrer l'état	met la VM en sommeil sur disque ; reprise à l'identique
Pause	gèle la VM en mémoire

ADDITIONS INVITÉ ET INTÉGRATION

- Apportent : écran redimensionnable, presse-papiers partagé, glisser-déposer, **dossiers partagés**, pilotes plus rapides.
- VirtualBox : menu **Périphériques** → **Insérer l'image CD des Additions invité** ; sous Linux, installer d'abord les outils de compilation et les entêtes du noyau.
- Dossier partagé sous Linux : `/media/sf_nom`, accessible au groupe `vboxsf` : `sudo usermod -aG vboxsf $USER`, puis rouvrir la session.
- Équivalents : `open-vm-tools` (VMware), `qemu-guest-agent` et `spice-vdagent` (KVM).
- **!** Presse-papiers, glisser-déposer et dossiers partagés **percent l'isolement** : les couper pour un laboratoire.

ÉCHANGER DES FICHIERS AVEC LA VM

- **Dossier partagé** (additions invité) : le plus simple.
- **SSH** par la redirection NAT : `scp -P 2222 f utilisateur@127.0.0.1` (serveur SSH à installer dans l'invité : `sudo apt install openssh-server`).
- **Glisser-déposer, presse-papiers** : pratiques au quotidien.
- **Clé USB** attachée à la VM (menu **Périphériques** → **USB**) ou image ISO de fichiers.

DISQUES VIRTUELS

VDI	format natif de VirtualBox
VMDK	VMware ; lu aussi par VirtualBox et QEMU
VHD · VHDX	Hyper-V ; VHDX aussi pour le disque de WSL 2
qcow2	QEMU / KVM : dynamique, instantanés internes
raw · img	image brute, octet par octet
OVA · OVF	machine entière exportée (disques + réglages) : format d'échange entre hyperviseurs
Dynamique	grossit à l'usage, ne rétrécit pas seul ; économe
Fixe	taille réservée d'emblée ; un peu plus rapide

Convertir : `qemu-img convert -f vmdk -O qcow2 a.vmdk b.qcow2`.
Agrandir le disque ne suffit pas : étendre ensuite la partition **dans** l'invité.

INSTANTANÉ, CLONE, EXPORT, SAUVEGARDE

Instantané	fige disque (et mémoire si la VM tourne) ; retour en quelques secondes ; dépend du disque d'origine
Clone complet	copie indépendante de la VM
Clone lié	partage le disque du parent : léger, mais inutilisable sans lui
Export OVA	un seul fichier pour déplacer la VM vers un autre poste ou hyperviseur
Sauvegarde	copie du dossier de la VM éteinte sur un autre support

! Un instantané **n'est pas une sauvegarde** : si le disque d'origine est perdu, l'instantané l'est aussi. Supprimer un instantané **fusionne** les différences sans toucher à l'état actuel. Éviter les longues chaînes.

MONTER UN LABORATOIRE

- Deux VM ou plus en **réseau interne** nommé (ex. **labo**) : elles se voient, et rien d'autre.
- Pas de DHCP en réseau interne : adresses fixes (ex. **10.10.10.1**, **10.10.10.2**).
- Besoin d'Internet pour installer ? Ajouter **temporairement** une 2^e carte en NAT, puis la retirer.
- **!** Une machine **volontairement vulnérable** ne va jamais en mode pont.

FICHIERS D'UNE VM (VIRTUALBOX)

VirtualBox VMS/	dans le dossier personnel ; un sous-dossier par VM
vm.vbox · vm.vdi	réglages (XML) · disque virtuel
Snapshots/	disques différentiels, états mémoire
Logs/VBox.log	à lire quand la VM refuse de démarrer
! Déplacer, renommer : par VirtualBox, jamais à la main.	

UNE VM QUI RAME ?

- Virtualisation matérielle **activée** (page 1) et **additions invité** installées.
- Disque virtuel sur un **SSD** ; hôte pas à court de mémoire.
- Ne pas **surallouer** : trop de processeurs ou de mémoire ralentit l'hôte... et la VM.
- Invité Linux : carte réseau **paravirtualisée** (`virtio-net`).

3 · Outils : VirtualBox, KVM, WSL, Hyper-V

VIRTUALBOX EN LIGNE DE COMMANDE : VBOXMANAGE

Chaque commande ci-dessous commence par **VBoxManage** :

VBoxManage list vms.

list vms	VM enregistrées (list runningvms : en marche)
list ostypes	identifiants de systèmes pour --ostype
showvminfo vm	configuration complète
startvm vm --type headless	démarrer sans fenêtre (serveur)
controlvm vm acpipowerbutton	arrêt propre
controlvm vm savestate	enregistrer l'état et arrêter
controlvm vm poweroff	❗ extinction forcée
controlvm vm pause · resume	geler · reprendre

Instantanés

snapshot vm take nom	prendre (--description possible)
snapshot vm list	lister
snapshot vm restore nom	revenir (VM éteinte)
snapshot vm delete nom	supprimer (fusion)

Cloner, déplacer, supprimer

clonevm vm --name copie --register	clone complet, enregistré
export vm -o vm.ova	exporter en OVA
import fichier.ova	importer
unregistervm vm --delete	❗ supprime la VM et ses disques

Réglages (VM éteinte)

modifyvm vm --memory 4096 --cpus 2	mémoire en Mo, processeurs
modifyvm vm --nic1 nat	carte 1 : nat, bridged, intnet, hostonly, natnetwork, none
modifyvm vm --nic1 intnet --intnet1 labo	réseau interne nommé « labo »
modifyvm vm --natpf1 "ssh,tcp,,2222,,22"	redirection NAT : port 2222 de l'hôte → port 22 de l'invité
sharedfolder add vm --name nom --hostpath chemin --automount	dossier partagé, monté automatiquement
modifymedium disk f.vdi --compact	compacter un disque dynamique (après remise à zéro de l'espace libre dans l'invité)

Sous Windows, **VBoxManage.exe** se trouve dans **C:\Program Files\Oracle\VirtualBox** (souvent absent du **PATH**).

CRÉER UNE VM SANS L'INTERFACE

```
# chaque ligne commence par VBoxManage
createvm --name labo --ostype Ubuntu_64 --register
modifyvm labo --memory 4096 --cpus 2 --nic1 nat
createmedium disk --filename labo.vdi --size 25600
storagectl labo --name SATA --add sata
storageattach labo --storagectl SATA --port 0 --device 0
--type hdd --medium labo.vdi # suite de la ligne
storageattach labo --storagectl SATA --port 1 --device 0
--type dvddrive --medium systeme.iso # suite
startvm labo

--size en Mo : 25600 = 25 Go. Identifiant exact du système : list ostypes.
```

TOUCHE HÔTE DE VIRTUALBOX

Ctrl droit	touche hôte : libère souris et clavier
Hôte+F	plein écran · Hôte+C mode redimensionné
Hôte+Suppr	envoie Ctrl+Alt+Suppr à l'invité
Hôte+T	prendre un instantané
Hôte+H	arrêt ACPI

KVM, QEMU, LIBVIRT (HÔTE LINUX)

virsh list --all	toutes les VM, allumées ou non
virsh start vm	démarrer
virsh shutdown vm	arrêt propre (ACPI)
virsh destroy vm	❗ extinction forcée — ne supprime rien, malgré le nom
virsh undefine vm	retirer la VM ; disques conservés sauf --remove-all-storage
virsh autostart vm	démarrer avec l'hôte
virsh console vm	console série ; sortir : [Ctrl+]
virsh dominfo vm	état, mémoire, processeurs
virsh snapshot-create-as vm nom	instantané
virsh snapshot-list vm	lister
virsh snapshot-revert vm nom	revenir
virsh net-list --all	réseaux virtuels
qemu-img create -f qcow2 d.qcow2 20G	créer un disque de 20 Go
qemu-img info d.qcow2	format, taille réelle et virtuelle
qemu-img resize d.qcow2 +10G	agrandir de 10 Go

Interface graphique : **virt-manager**. Gérer sans **sudo** : **sudo usermod -aG libvirt \$USER**. Noms des paquets (QEMU, libvirt) : selon la distribution.

WSL 2 (POWERSHELL OU INVITE DE COMMANDES)

ws1 --install	installer WSL et Ubuntu (administrateur, redémarrage)
ws1 -l -o	distributions disponibles (--list --online)
ws1 --install distribution	en installer une autre
ws1 -l -v	installées, état, version 1 ou 2
ws1 -d distribution	ouvrir une distribution précise
ws1 --shutdown	arrêter toutes les distributions et la VM
ws1 -t distribution	arrêter une distribution (--terminate)
ws1 --export d f.tar	sauvegarder une distribution
ws1 --import nom dossier f.tar	la restaurer ailleurs
ws1 --unregister d	❗ supprime la distribution et toutes ses données , sans retour
ws1 --update · --version · --status	mettre à jour · versions · configuration

```
# %UserProfile%\wslconfig - Limiter la VM de WSL 2
[ws12]
memory=4GB
processors=2
```

Appliquer : **ws1 --shutdown**. Fichiers Windows vus de Linux : **/mnt/c** ; fichiers Linux vus de Windows : **\\ws1.localhost\distribution**. Travailler dans le système de fichiers Linux : bien plus rapide.

HYPER-V (WINDOWS PRO, ENTREPRISE, ÉDUCATION)

Get-VM	lister les VM
Start-VM -Name vm	démarrer
Stop-VM -Name vm	arrêt propre (-TurnOff : forcé)
Checkpoint-VM -Name vm -SnapshotName nom	point de contrôle (= instantané)
Restore-VMSnapshot -VMName vm -Name nom	revenir

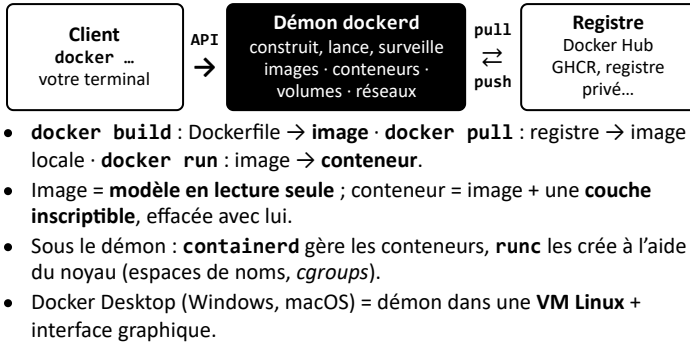
Activation : *Fonctionnalités de Windows* → **Hyper-V**. Commandes PowerShell en administrateur.

Mémento — VM & Docker · v1.1 · septembre 2026

didierdambreville.github.io · 3 / 8

4 · Docker : l'essentiel

COMMENT DOCKER FONCTIONNE



LES BRIQUES D'UN CONTENEUR LINUX

Espaces de noms	ce que le processus voit : ses processus, son réseau, ses montages, son nom d'hôte
Groupes de contrôle	ce qu'il peut consommer : mémoire, processeur, disque, nombre de processus (<i>cgroups</i>)
Capacités	les pouvoirs de root découpés ; Docker n'en laisse qu'une partie
seccomp, AppArmor, SELinux	filtrent les appels système et les accès
Couches (OverlayFS)	image en lecture seule, partagée + couche inscriptible propre au conteneur

Aucune « machine » : un conteneur est un **processus ordinaire** de l'hôte, simplement cloisonné.

ANATOMIE DE DOCKER RUN

docker run	-d	--name web	-p 8080:80
COMMANDE	EN ARRIÈRE-PLAN	NOM	PORT HÔTE:CONTENEUR
nginx:alpine			
IMAGE:ÉTIQUETTE			

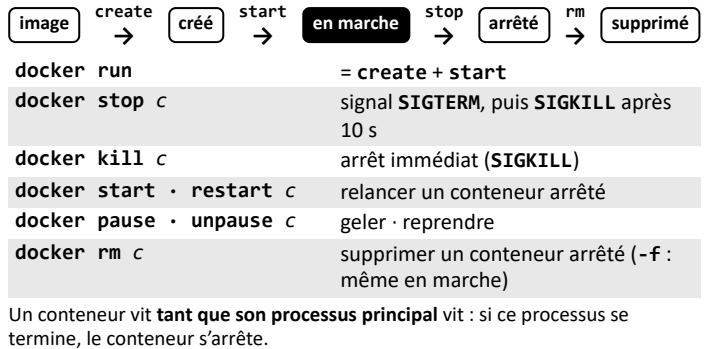
Puis ouvrir <http://localhost:8080>. Tout ce qui suit l'image remplace sa commande par défaut : **docker run --rm alpine ls /etc**.

OPTIONS ESSENTIELLES DE DOCKER RUN

-d	détaché (arrière-plan) ; affiche l'identifiant
-it	interactif avec terminal : docker run -it alpine sh
--rm	supprimer le conteneur à l'arrêt
--name nom	nom lisible (sinon nom aléatoire)
-p 8080:80	publier : port 8080 de l'hôte → 80 du conteneur
-p 127.0.0.1:8080:80	publier pour l'hôte seulement
-e VAR=valeur	variable d'environnement (--env-file f)
-v volume:/chemin	monter un volume nommé
-v "\$(pwd)":/app	monter le dossier courant (Bash)
-w /app	dossier de travail
--restart unless-stopped	relancer après panne ou redémarrage
--memory 512m --cpus 1.5	limiter mémoire et processeur
--user 1000:1000	s'exécuter sans être root
--network réseau	rejoindre un réseau
--entrypoint sh	remplacer le point d'entrée

Formes longues équivalentes : **docker container run = docker run**, **docker container ls = docker ps**, **docker image ls = docker images**.

CYCLE DE VIE D'UN CONTENEUR



OBSERVER ET INTERVENIR

docker ps	conteneurs en marche (-a : tous)
docker logs -f --tail 100 c	suivre le journal (--since 10m)
docker exec -it c sh	ouvrir un shell dans un conteneur en marche (bash s'il existe)
docker stats	processeur, mémoire, réseau en direct
docker top c	processus du conteneur
docker inspect c	tout le détail en JSON
docker inspect -f '{{.State.Status}}' c	extraire un champ
docker port c	ports publiés
docker cp c:/chemin ./	copier depuis (ou vers) un conteneur
docker diff c	fichiers modifiés depuis l'image
docker events	événements du démon en direct

PREMIERS PAS

```
docker run hello-world
docker run -it --rm alpine sh # shell jetable
docker run -d --name web -p 8080:80 nginx:alpine
docker ps # web est « Up »
docker logs web # journal d'accès
docker stop web && docker rm web # ménage
```

INSTALLER ET VÉRIFIER

Linux	dépôt officiel de Docker, puis paquets docker-ce , docker-ce-cli , containerd.io , docker-buildx-plugin , docker-compose-plugin
sudo docker run hello-world	test : téléchargement et lance une image d'essai
docker version · info	versions du client et du démon · configuration
sudo systemctl enable --now docker	démarrer le démon, et à chaque démarrage

❗ Désinstaller d'abord les paquets non officiels (**docker.io**, **podman-docker**...). Le script **get.docker.com** : réservé aux essais, pas à la production.

FILTRE ET FORMATER

docker ps -q	identifiants seuls
docker ps -a -f status=exited	filtrer : conteneurs arrêtés
docker ps -f name=web	filtrer par nom
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"	colonnes choisies
docker stop \$(docker ps -q)	arrêter tout ce qui tourne (Bash)

Un préfixe unique de l'identifiant suffit : **docker stop 3f2a**.

5 · Images et Dockerfile

NOM COMPLET D'UNE IMAGE

docker.io/ **library/** **postgres** **:17** **@sha256:...**

REGISTRE ESPACE DE NOMS DÉPÔT ÉTIQUETTE EMPREINTE

- Registre par défaut : Docker Hub · espace **library** = images **officielles**.
- Sans étiquette : **latest** — ce n'est **pas** forcément la plus récente, juste l'étiquette par défaut.
- L'empreinte **@sha256:** désigne un contenu **immuable** ; une étiquette peut être déplacée.

GÉRER LES IMAGES

docker pull <i>image:tag</i>	télécharger
docker images	images locales (docker image ls)
docker images -f dangling=true	images « pendantes », sans nom
docker rmi <i>image</i>	supprimer (si aucun conteneur ne l'utilise)
docker tag <i>src</i> <i>registre/nom:tag</i>	donner un autre nom
docker login <i>registre</i>	s'authentifier (Docker Hub par défaut)
docker push <i>registre/nom:tag</i>	publier
docker history <i>image</i>	couches et instructions qui les ont créées
docker save -o <i>f.tar</i> <i>image</i>	exporter une image dans un fichier...
docker load -i <i>f.tar</i>	... et la charger sur une autre machine
docker search <i>mot</i>	chercher sur Docker Hub

❗ **docker export / import** travaillent sur un **conteneur** : système de fichiers aplati, sans couches ni réglages. **docker commit** fige un conteneur bricolé à la main : préférer un Dockerfile, reproductible.

UN DOCKERFILE COMMENTÉ

```
# syntax=docker/dockerfile:1
FROM python:3.14-slim          # base officielle, version fixée
WORKDIR /app                  # dossier de travail
COPY requirements.txt .        # 1. Les dépendances...
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                      # 2. ...puis le code
RUN useradd --create-home appli
USER appli                    # ne pas rester root
ENV PORT=8000
EXPOSE 8000                   # documente le port
CMD ["python", "app.py"]      # commande par défaut
```

docker build -t appli:1.0 . construire ; le **.** final = **contexte** envoyé au démon

-f <i>autre.Dockerfile</i>	autre fichier que Dockerfile
--no-cache	tout reconstruire
--build-arg <i>VAR=val</i>	valeur d'un ARG
--target <i>étape</i>	s'arrêter à une étape nommée
--platform <i>linux/amd64</i>	architecture visée (PC ou ARM)

PUBLIER UNE IMAGE

```
docker login                  # Docker Hub
docker tag appli:1.0 compte/appli:1.0
docker push compte/appli:1.0
```

Autre registre : préfixer par son adresse, **registre/appli:1.0**. ❗ Un dépôt public est visible de tous.

TESTER LA SANTÉ D'UN SERVICE

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -fs http://localhost:8000/ || exit 1

docker ps
```

affiche alors (**healthy**), (**unhealthy**) ou (**health: starting**).
❗ L'outil appelé (**curl**) doit exister dans l'image.

INSTRUCTIONS

FROM	image de départ ; FROM ... AS <i>nom</i> nomme une étape
RUN	exécute une commande à la construction ; crée une couche
COPY	copie depuis le contexte (--chown=u:g)
ADD	comme COPY + archives et URL : préférer COPY
WORKDIR	dossier courant des instructions suivantes
ENV	variable conservée dans l'image
ARG	variable de construction seulement
EXPOSE	documente un port ; ne publie rien
USER	utilisateur des instructions suivantes et du conteneur
VOLUME	point de montage prévu pour des données
ENTRYPOINT	programme toujours lancé
CMD	commande ou arguments par défaut
HEALTHCHECK	test de bonne santé périodique
LABEL	métadonnées : auteur, version, licence

CMD, ENTRYPOINT ET LEURS FORMES

CMD ["app", "-v"]	forme exec (JSON) : le programme est le processus 1 et reçoit les signaux → à préférer
CMD <i>app -v</i>	forme shell : lancé par /bin/sh -c ; ❗ docker stop attend souvent les 10 s
ENTRYPOINT + CMD	ENTRYPOINT ["ping"] + CMD ["localhost"] ; docker run <i>img</i> exemple.org remplace seulement CMD

COUCHES ET CACHE

- Chaque instruction produit une couche ; si l'une change, **toutes les suivantes** sont reconstruites.
- Ordre : ce qui change **rarement** en haut (système, dépendances), le code **en bas**.
- Installer et nettoyer dans la même couche, sinon les fichiers supprimés pèsent encore :

```
RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/*
```

CONSTRUCTION EN PLUSIEURS ÉTAPES

```
FROM node:22 AS build          # 1. outils complets
WORKDIR /src
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build              # produit /src/dist
FROM nginx:alpine              # 2. image finale légère
COPY --from=build /src/dist /usr/share/nginx/html
```

L'image finale ne contient que le résultat : ni compilateur, ni sources, ni dépendances de développement.

.DOCKERIGNORE ET BONNES PRATIQUES

```
.git
node_modules
*.log
.env
```

secrets : jamais dans le contexte

- Image de base **officielle** et **minimale** (**-slim**, **alpine**, **distroless**), version fixée.
- **Un processus** principal par conteneur.
- ❗ Aucun secret dans l'image : **ENV**, **ARG** et fichiers copiés restent lisibles (**docker history**, couches).
- Utilisateur non root ; **HEALTHCHECK** pour les services.

6 · Données et réseau

OÙ VIVENT LES DONNÉES ?

TYPE	PRINCIPE	POUR
Volume nommé	géré par Docker, hors du conteneur ; survit à <code>docker rm</code>	données d'une base, fichiers d'une application
Montage lié (bind)	un dossier de l'hôte vu dans le conteneur	développement : code source édité sur l'hôte
tmpfs	en mémoire vive, effacé à l'arrêt (Linux)	fichiers temporaires, sensibles
Couche du conteneur	écrite dans le conteneur lui-même	❗ perdue avec <code>docker rm</code>

SYNTAXES

- `-v donnees:/var/lib/data` volume nommé (créé s'il n'existe pas)
- `-v /srv/site:/site:ro` montage lié, en lecture seule
- `-v "$(pwd)":/app` dossier courant · PowerShell : `"${PWD}:/app"`
- `--mount type=volume,src=donnees,dst=/data` forme longue, explicite
- `--mount type=bind,src=/srv/site,dst=/site` montage lié ; refuse un chemin d'hôte absent
- `--tmpfs /tmp` dossier temporaire en mémoire
- `-v` crée silencieusement un dossier d'hôte absent (appartenant à root) ; `--mount` signale l'erreur.
- Un **volume nommé vide** reçoit le contenu du dossier de l'image ; un **montage lié le masque**.
- Droits : le conteneur écrit avec son identifiant (UID) ; aligner avec `--user`.

GÉRER LES VOLUMES

- `docker volume create v` créer
- `docker volume ls` lister
- `docker volume inspect v` emplacement réel (Mountpoint)
- `docker volume rm v` ❗ supprimer, avec les données
- `docker volume prune` volumes anonymes inutilisés
- `docker volume prune -a` ❗ tous les volumes inutilisés

SAUVEGARDER ET RESTAURER UN VOLUME

```
# sauvegarde : archive dans le dossier courant
docker run --rm -v donnees:/source:ro \
  -v "$(pwd)":/sauv alpine \
  tar czf /sauv/donnees.tar.gz -C /source .
# restauration dans un volume (vide)
docker run --rm -v donnees:/cible \
  -v "$(pwd)":/sauv alpine \
  tar xzf /sauv/donnees.tar.gz -C /cible
```

Arrêter d'abord le conteneur qui écrit dans le volume. Pour une base de données, préférer son outil d'export (`pg_dump`, `mysqldump`).

RÉGLAGES DU DÉMON : /ETC/DOCKER/DAEMON.JSON

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "data-root": "/srv/docker",
  "default-address-pools": [
    { "base": "10.200.0.0/16", "size": 24 } ]
}
```

- ❗ Journaux **json-file sans rotation** par défaut : un conteneur bavard remplit le disque. Les options valent pour les conteneurs **créés ensuite**.
- data-root** : déplacer le stockage de Docker ; **default-address-pools** : si **172.17.x.x** entre en conflit avec le réseau local.
- Appliquer : `sudo systemctl restart docker` · Docker Desktop : **Settings → Docker Engine**.

PILOTES DE RÉSEAU

- bridge** (défaut) réseau privé sur l'hôte ; ports publiés avec `-p`
- bridge défini par l'utilisateur** idem + **résolution par nom** : `db`, `cache`... → à utiliser
- host** partage la pile réseau de l'hôte, sans isolement (Linux)
- none** aucun réseau
- overlay** réseau commun à plusieurs hôtes (Swarm)
- macvlan** adresse propre sur le réseau local, comme une machine physique

COMMANDES RÉSEAU

- `docker network ls` lister
- `docker network create r` créer un réseau *bridge*
- `docker run --network r ...` lancer dans ce réseau
- `docker network connect r c` brancher un conteneur existant
- `docker network inspect r` conteneurs et adresses
- `docker network rm · prune` supprimer un réseau · les inutilisés

```
docker network create appnet
docker run -d --name cache --network appnet redis:alpine
docker run -it --rm --network appnet redis:alpine \
  redis-cli -h cache ping # → PONG
```

Le nom du conteneur (`cache`) sert de nom d'hôte. Sur le réseau **bridge par défaut**, cette résolution ne fonctionne pas.

PUBLIER UN PORT

- `navigateur` `localhost:8080` → `hôte` `-p 8080:80` → `conteneur :80`
- `-p 8080:80` toutes les interfaces de l'hôte (`0.0.0.0` et `:::`) : visible du réseau
- `-p 127.0.0.1:8080:80` depuis l'hôte seulement
- `-p 5353:53/udp` port UDP
- `-P` tous les ports **EXPOSE** vers des ports hôte au hasard

- ❗ Linux : les ports publiés **contournent le pare-feu ufw** ; publier sur **127.0.0.1** ce qui ne doit pas sortir.
- Dans un conteneur, **localhost = le conteneur lui-même**, pas l'hôte.
- Joindre l'hôte : `host.docker.internal` (Docker Desktop ; sous Linux : `--add-host=host.docker.internal:host-gateway`).
- Entre conteneurs d'un même réseau : le port **interne** (`db:5432`), sans publication.

DIAGNOSTIQUER LE RÉSEAU

- `docker port c` publications effectives
- `docker network inspect r` qui est branché, avec quelle adresse
- `docker exec c getent hosts db` le nom `db` se résout-il ? (ou `nslookup db`, selon l'image)
- `docker exec c wget -qO- http://web:8080` le service répond-il ? (`curl` s'il existe)
- `sudo ss -tlnp` hôte Linux : quel programme écoute sur quel port ?
- `docker run --rm -it --network container:c alpine sh` shell de dépannage **dans la pile réseau** du conteneur, quand son image n'a aucun outil

OÙ DOCKER RANGE TOUT

- Linux : **/var/lib/docker** — images, conteneurs, volumes. ❗ N'y rien modifier à la main.
- Docker Desktop : dans le disque virtuel de sa VM.
- `docker system df -v` : détail par image, conteneur, volume.
- `~/docker/config.json` : réglages du client ; ❗ **docker login** y range les identifiants simplement encodés, sans chiffrement, faute d'assistant d'identifiants (*credential helper*).

7 · Docker Compose

UN COMPOSE.YAML COMMENTÉ

```
services:
  web:                                # nom du service = nom d'hôte
    build: .                          # image construite ici
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      DB_HOST: db                    # Le service « db »
    env_file: .env                  # variables injectées
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

db:
  image: postgres:17                # version fixée
  environment:
    POSTGRES_USER: appli
    POSTGRES_DB: appli
    POSTGRES_PASSWORD_FILE: /run/secrets/db_mdp
  secrets: [db_mdp]
  volumes:
    - donnees:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U appli"]
    interval: 5s
    retries: 5

volumes:
  donnees:                          # volume nommé

secrets:
  db_mdp:
    file: ./db_mdp.txt              # hors dépôt Git !
```

Le mot de passe est lu dans un **fichier** monté en **/run/secrets/**, pas écrit en clair dans le YAML. **!** YAML : indentation par **espaces** (jamais de tabulation) ; mettre les ports entre guillemets.

DÉMARRER DANS L'ORDRE

- **depends_on** seul garantit l'**ordre de lancement**, pas que le service soit prêt.
- Avec **condition: service_healthy**, Compose attend que le **healthcheck** du service réussisse.
- L'application doit tout de même savoir **réessayer** une connexion perdue.

DE DOCKER RUN À COMPOSE

DOCKER RUN	COMPOSE.YAML
-p 8080:80	ports: ["8080:80"]
-v donnees:/data	volumes: ["donnees:/data"] + section volumes :
-e VAR=val	environment: puis VAR: val
--env-file .env	env_file: .env
--restart unless-stopped	restart: unless-stopped
--network appnet	networks: [appnet]
--memory 512m	mem_limit: 512m
--name web	container_name: web (rarement utile)
image cmd	command: cmd

METTRE À JOUR L'APPLICATION

```
docker compose pull      # nouvelles images
docker compose up -d     # recrée ce qui a changé
docker image prune       # anciennes images
```

- Lire les **notes de version** ; sauvegarder les volumes **avant**.
- **!** Base de données : changer de version **majeure** (PostgreSQL 17 → 18) exige une **migration** des données.
- Image construite localement : **docker compose up -d --build**.

PRINCIPE

- Un fichier décrit **toute l'application** : services, réseaux, volumes, secrets.
- Nom par défaut : **compose.yaml** (préféré) ; **docker-compose.yaml** reste lu.
- Commande actuelle : **docker compose** (avec une **espace**) ; l'ancien **docker-compose** (tiret) est obsolète.
- La clé **version** : en tête de fichier est **obsolète** : Compose l'ignore et avertit.
- Un réseau **projet_defaut** relie les services : chacun joint les autres **par son nom**.
- Projet = nom du dossier, préfixe des conteneurs et volumes (**-p nom** pour le changer).

COMMANDES

Chaque commande commence par **docker compose** :

docker compose up -d.

up -d	créer et démarrer en arrière-plan
up -d --build	reconstruire les images d'abord
ps	état des services
logs -f service	suivre les journaux
exec service sh	shell dans un service en marche
run --rm service cmd	tâche ponctuelle dans un conteneur neuf
restart service	redémarrer
stop · start	arrêter · relancer sans supprimer
pull	mettre à jour les images
config	fichier final, variables résolues : valider
down	supprimer conteneurs et réseau ; volumes gardés
down -v	! supprime aussi les volumes : données perdues
-f f.yaml ...	utiliser un autre fichier

CLÉS UTILES D'UN SERVICE

image · build	image existante · à construire (build: ./dossier)
ports	publication " hôte:conteneur "
volumes	volume nommé ou montage lié ./src:/app
environment · env_file	variables · fichier de variables
depends_on	ordre de démarrage (+ condition)
healthcheck	test de santé
restart	no, always, on-failure, unless-stopped
command · entrypoint	remplacer CMD · ENTRYPOINT
networks	réseaux rejoints
profiles	service facultatif : docker compose --profile p up
deploy.resources	limites de mémoire et de processeur

VARIABLES ET FICHIERS

- **.env** du dossier du projet : lu **automatiquement** pour remplacer **\${VAR}** dans le YAML.
- **\${VAR:-défaut}** valeur par défaut · **\${VAR:?message}** erreur si absente.
- **env_file** : injecte des variables **dans le conteneur** : ce n'est pas le même mécanisme.
- **compose.override.yaml**, s'il existe, est **fusionné** automatiquement : réglages de développement.
- **!** **.env** et fichiers de secrets : dans **.gitignore** et **.dockerignore**.

8 · Sécurité, ménage, dépannage

DURCIR UN CONTENEUR

- `--user 1000:1000` ne pas tourner en root
- `--read-only --tmpfs /tmp` système de fichiers en lecture seule
- `--cap-drop ALL` retirer les privilèges du noyau (`--cap-add` au cas par cas)
- `--security-opt no-new-privileges` interdire l'élévation (`setuid`)
- `--memory` · `--cpus` · borner les ressources
- `--pids-limit`
- `-p 127.0.0.1:...` ne rien exposer sans raison
- `--network none` couper le réseau
- ❗ Jamais `--privileged` ni `-v /var/run/docker.sock:...` sans comprendre : c'est donner l'hôte.
- ❗ Le groupe **docker** donne des droits **équivalents à root** sur l'hôte (documentation officielle). Alternatives : mode *rootless*, Podman.
- Images officielles ou d'éditeurs vérifiés ; version fixée ; les mettre à jour ; analyser les vulnérabilités (`docker scout cves image`, Trivy, Gripe).
- Secrets : fichiers montés (`secrets:`), jamais dans l'image ni dans le dépôt.

EXÉCUTER UN PROGRAMME DOUTEUX

- VM **dédiée**, jamais le poste principal ni un conteneur : le noyau partagé rend l'évasion plus plausible.
- Instantané **propre** avant, retour à cet instantané après.
- Réseau **interne** ou **non connecté** ; pas de dossier partagé, de presse-papiers ni de glisser-déposer.
- ❗ **WSL n'est pas une frontière de sécurité** : Linux y lit et écrit les fichiers Windows (`/mnt/c`).
- Idéalement : inspection du code **avant** toute exécution.

SÉCURITÉ DES VM

- Mettre à jour l'**hyperviseur** : des failles d'évasion de VM existent.
- Un instantané pris VM allumée contient la **mémoire** : mots de passe, clés... le protéger comme la VM.
- Données sensibles : chiffrer le disque **dans** l'invité (LUKS, BitLocker).
- N'importer que des OVA de source sûre ; couper dossiers partagés, presse-papiers et USB quand ils ne servent pas.

SERVICE EN PRODUCTION : LISTE DE CONTRÔLE

- Image à version fixée, base officielle, **reconstruite** régulièrement.
- Utilisateur non root, privilèges retirés (`--cap-drop`).
- healthcheck** et **restart: unless-stopped**.
- Limites de mémoire et de processeur ; journaux bornés (`max-size`).
- Ports liés à `127.0.0.1`, derrière un **proxy inverse** en HTTPS.
- Volumes sauvegardés et **restauration testée**.
- Secrets hors de l'image et du dépôt ; mises à jour surveillées.

DOCKER ↔ PODMAN

- Mêmes commandes : `podman run`, `podman ps`, `podman build...` (`alias docker=podman`).
- Sans démon** et **sans root** par défaut ; notion de *pod* (groupe de conteneurs).
- ❗ Donner les noms d'images complets : `docker.io/library/nginx:alpine`.

POUR ALLER PLUS LOIN

- docs.docker.com : guides, référence de la CLI, de Compose et du Dockerfile.
- [virtualbox.org/manual](https://www.virtualbox.org/manual) : manuel de VirtualBox, dont VBoxManage.
- learn.microsoft.com/windows/wsl : WSL.
- libvirt.org · qemu.org : KVM et QEMU.
- Pratiquer : tout refaire dans une VM jetable, instantané à l'appui.

FAIRE LE MÉNAGE

- `docker system df` espace occupé : images, conteneurs, volumes, cache
- `docker container prune` conteneurs arrêtés
- `docker image prune` images pendantes (`-a` : toutes les inutilisées)
- `docker builder prune` cache de construction
- `docker volume prune` volumes anonymes (`-a` : tous les inutilisés)
- `docker system prune` conteneurs arrêtés, réseaux inutilisés, images pendantes, cache
- `docker system prune -a --volumes` ❗ grand ménage ; volumes **nommés** épargnés

Tout **prune** demande confirmation ; `-f` la saute. Lire la liste **avant** de valider.

MESSAGES FRÉQUENTS

- `permission denied while trying to connect to the Docker daemon socket` utilisateur hors du groupe **docker** : **sudo**, ou ajout au groupe puis nouvelle session
- `Cannot connect to the Docker daemon ... Is the docker daemon running?` démon arrêté : `sudo systemctl start docker`, ou lancer Docker Desktop
- `port is already allocated` port de l'hôte déjà pris : en choisir un autre
- `Conflict. The container name "/web" is already in use` `docker rm web` ou autre `--name`
- `pull access denied ... repository does not exist` nom mal orthographié ou image privée (**docker login**)
- `no space left on device` `docker system df`, puis ménage
- `exec format error` image d'une autre architecture (ARM ↔ PC) : `--platform`
- `VT-x is disabled in the BIOS ... 0x80370102 (WSL)` virtualisation matérielle désactivée : l'activer dans l'UEFI

CODES DE SORTIE D'UN CONTENEUR

- 0** fin normale
- 1** erreur de l'application : lire **docker logs**
- 125** `docker run` lui-même a échoué (option, image)
- 126 · 127** commande non exécutable · introuvable
- 137** tué par **SIGKILL** (128 + 9) : souvent **mémoire** dépassée → `docker inspect -f '{{.State.OOMKilled}}' c`
- 143** arrêté par **SIGTERM** (128 + 15), par exemple `docker stop`

Démarche : `docker ps -a` → `docker logs` → `docker inspect` → `docker exec -it c sh` ; conteneur qui s'arrête aussitôt : `docker run -it --rm --entrypoint sh image` pour regarder dedans.

GLOSSAIRE ANGLAIS → FRANÇAIS

<i>bind mount</i>	montage lié	<i>layer</i>	couche
<i>bridged</i>	accès par pont	<i>linked clone</i>	clone lié
<i>build context</i>	contexte	<i>namespace</i>	espace de noms
<i>checkpoint</i>	point de contrôle	<i>prune</i>	purger
<i>dangling image</i>	image pendante	<i>registry</i>	registre
<i>digest</i>	empreinte	<i>restart policy</i>	politique de redémarrage
<i>guest additions</i>	additions invité	<i>rootless</i>	sans root
<i>headless</i>	sans fenêtre	<i>snapshot</i>	instantané
<i>health check</i>	test de santé	<i>tag</i>	étiquette
<i>host-only</i>	réseau privé hôte	<i>volume</i>	volume